Week 2 Lab: Numerical and Process-Based Models

Rongling Liu (Student 482284924)

2026-08-10

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Part A: Tracer decay and step size

This part models tracer decay and examines how the decay rate and numerical time-step size affect the model results.

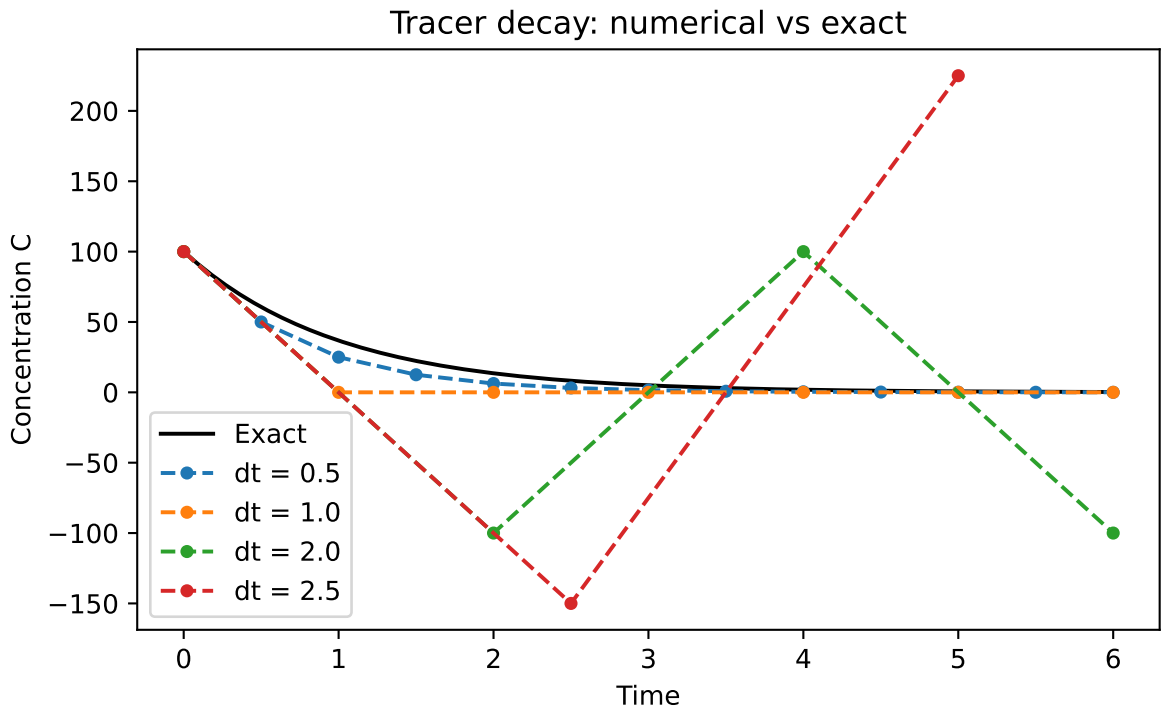
```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)
```

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.5, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("Tracer decay: numerical vs exact")
ax.legend()
plt.show()
```



Answer to task A1: The decay rate k controls how quickly the tracer concentration decreases, with a larger k causing faster decay and a smaller k causing slower decay.

Answer to task A2: The numerical curve first visibly disagreed with the exact solution at $dt = 2.0$ days.

Answer to task A3: The largest time step that still gives a non-negative curve is $dt = 1.0$.

Answer to task A4: When $k\,dt > 1$, the update rule $C_{\text{new}} = C_{\text{old}}(1 - k\,dt)$ makes $1 - k\,dt$ negative. This causes the numerical model to remove more tracer than is present in one step, so the concentration overshoots below zero. ## Part B: Reservoir box model

```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6        # lake volume (m^3)
Q = 5.0e3        # river inflow (m^3 per day)
C_in = 10.0      # incoming concentration (mg/L)
k = 0.05         # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0         # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0          # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V   # the rate of change
    C = C + dCdt * dt                           # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")
# ---- Analytical steady state ----
C_star = (Q * C_in) / (Q + k * V)

print(f"Analytical steady state: {C_star:.2f} mg/L")
print(f"Numerical final value: {history[-1]:.2f} mg/L")

fig, ax = plt.subplots(figsize=(6, 4))

for k_test in [0.02, 0.05, 0.1, 0.2]:
```

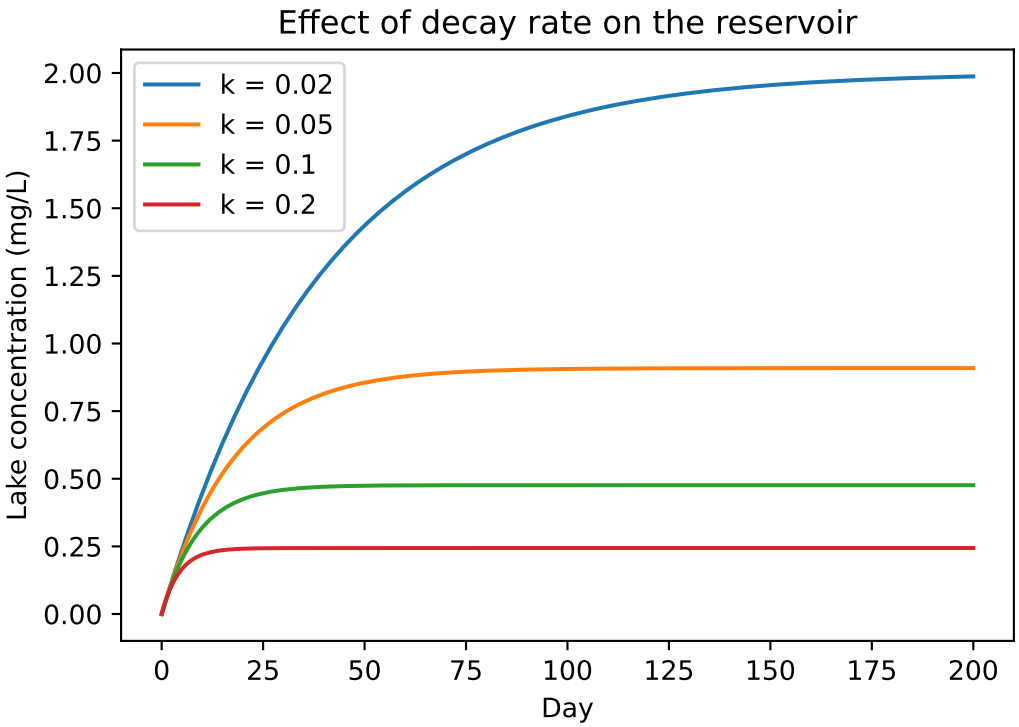
```
    C = 0.0
    history = [C]

    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k_test * V * C) / V
        C = C + dCdt * dt
        history.append(C)

    ax.plot(history, label=f"k = {k_test}")

ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Effect of decay rate on the reservoir")
ax.legend()
plt.show()
```

```
Started at 0.00 mg/L
After 10 days: 0.39 mg/L
After 50 days: 0.86 mg/L
After 200 days: 0.91 mg/L
Analytical steady state: 0.91 mg/L
Numerical final value: 0.91 mg/L
```



Answer to task B1: The analytical steady-state concentration is $C^* = 0.91$ mg/L, and the numerical model also approaches approximately 0.91 mg/L after 200 days.

Answer to task B2: Starting from $C = 20$ mg/L, the lake concentration decreases over time and approaches the same steady-state concentration of about 0.91 mg/L.

Answer to task B3: Doubling Q increases the steady-state concentration from about 0.91 to 1.67 mg/L because, in $C^* = QC_{in}/(Q + kV)$, increasing Q increases the numerator relative to the fixed decay term kV in the denominator.

Answer to task B4: Increasing k lowers the steady-state concentration and makes the system approach steady state more quickly.

Reflection

Since I have not studied physics for a long time, I found it difficult at first to understand some of the concepts and code used in the examples. I also sometimes forgot to reset the parameters to their default values when moving from one task to the next. However, this lab made me appreciate how Python can be used to solve numerical problems accurately even when the underlying concepts are challenging, and I enjoyed the process of coding more than I expected. Next time, I would spend more time understanding the logic behind each part of the code and check the parameter values carefully before starting a new task.